

Ссылки на задания, выложенные на реплит:

Задание 1 (калькулятор):

<https://replit.com/@evgeniiayd/lablonskaia-levghienii-IVT-12-LR-2-zadaniie-1-kalkuliator#main.py>

Задание 2 (отгадывание числа):

https://replit.com/@evgeniiayd/lablonskaia-levghienii-IVT-12-LR-1-zadaniie-2-otghadyvaniie#guess_number.py

Отчёт.

Задание 1.

1. Формулировка:

Создайте простую программу калькулятор, которая позволяет из функции `main()` ввести два числа и тип арифметической операции, а потом вычисляет результат. Свой код опубликуйте на `replit` и предоставьте ссылку в ответах на лабораторную работу в Moodle в документе-отчёте. Реализацию арифметических действий и вычисление результата с его возвратом сделайте в отдельной функции `calculate(...)`. Протестируйте свой калькулятор с помощью вызова нескольких своих простых функций `test_*`() с ключевым словом `assert` внутри. Обязательно напишите хорошую документацию к своему коду.

Модернизируйте калькулятор из задач 1.2 и 1.4 Лабораторной работы № 1. Добавьте к калькулятору такую настройку как точность вычислений, которая передаётся в виде `keyword` параметра `tolerance` со значением по умолчанию `1e-6`. На основе переданного значения этого параметра извлеките с помощью вычислений порядок этого значения (например 6 для `1e-6`) в виде отдельной функции `convert_precision`, вызываемой из `calculate`.

ЗадOCUMENTИРУЙТЕ `convert_precision` и дополните документацию к `calculate` в коде. Извлечённый порядок используйте для округления итогового результата в функции `calculate`. Покройте (напишите) дополнительными тестами `convert_precision` и `calculate` в связи с появлением `tolerance` с помощью пакета `pytest` или стандартных `unittest` Python по выбору.

Добавьте переменное количество неименованных аргументов (операндов, *args) после параметра action и перед keyword параметром tolerance. К списку поддерживаемых действий добавьте вычисление таких величин как среднее значение (medium), дисперсия (variance), стандартное отклонение (std_deviation), медиана (median, q2, второй квартиль) и межквартильный размах (q3 - q1, разница третьего и первого квартилей). Покройте новые реализованные функции и функцию calculate дополнительными юнит-тестами.

2. Математическая модель:

Среднее значение:

$$\bar{X} = \frac{\sum_{i=1}^n X_i}{n}$$

Дисперсия:

$$D = \frac{\sum (x_i - M)^2}{n - 1}$$

Стандартное отклонение:

$$\sigma = \sqrt{\frac{\sum (x_i - M)^2}{n - 1}}$$

3. Список идентификаторов:

Имя	Тип	Смысл
-----	-----	-------

x	float	Входное значение
y	float	Входное значение
z	str	Входное значение
tolerance	str -> float	Входное значение (по умолчанию: 1e-6)
arr	tuple	Входное значение
res	float или str	Вывод
order	int	Результат функции convert_precision
first	float	Первый параметр функции calculate - число
second	float	Второй параметр функции calculate - число
action	str	Третий параметр функции calculate - действие
*args	tuple	Кортеж дополнительных чисел для вычисления в функции calculate
precision	int	Результат функции convert_precision, который используется в функции calculate
list_of_args	list	Список кортежа *args
result	str или float	Результат функции calculate

sum_args	float	Промежуточная переменная
t	float	Промежуточная переменная
sum_variance	float	Промежуточная переменная
j	int	Счётчик цикла
i	int	Счётчик цикла
p	float	Промежуточная переменная
remaining_length	int	Промежуточная переменная
first_position	int	Промежуточная переменная
second_position	int	Промежуточная переменная
third_position	int	Промежуточная переменная
forth_position	int	Промежуточная переменная
q1	float	Промежуточная переменная
q3	float	Промежуточная переменная

4. Код программы:

```
main.py x calculate.py calculate_test.py
1 from calculate import calculate
2
3
4 1 usage
5 def main():
6     """
7     Function as a user's interface
8
9     Return an answer
10    """
11    print("Введите первое число: ")
12    x = float(input())
13    print("Введите второе число: ")
14    y = float(input())
15    print("Введите тип операции: ")
16    z = str(input())
17    print("Введите точность вычисления (по умолчанию 1e-6)")
18    tolerance = input()
19    print("Введите дополнительные числа через запятую(при необходимости)")
20    arr = input()
21    if arr != "":
22        arr = tuple(map(float, arr.split(',')))
23    if not tolerance:
24        res = calculate(x, y, z, *args: arr)
25    else:
26        res = calculate(x, y, z, *args: arr, tolerance=tolerance)
27    print("Ответ: ", res)
28
29 if __name__ == '__main__':
30     main()
```

```
1 usage
1 def convert_precision(tolerance):
2     """
3     Argument:
4         tolerance -- precision for calculation answer
5
6     Return a number of digits after a coma
7     """
8     tolerance = float(tolerance)
9     if (tolerance <= 0) or (tolerance >= 1):
10         return "Неверная точность"
11     order = 0
12     while tolerance != 1:
13         tolerance = tolerance * 10
14         order += 1
15     return order
16
17
18 24 usages
19 def calculate(first, second, action, *args, tolerance=1e-6):
20     """
21     Execute calculation such as: addition(+), subtraction(-), multiplication(*),
22     division(/), exponentiation(^), medium, variance, std_deviation, median,
23     interquartile_range
24
25     Arguments:
26         first -- first number, float
27         second -- second number, float
28         action -- action indication, string
29         args -- more numbers (unnecessary)
30         tolerance -- degree of accuracy
```



```

89     else:
90         remaining_length = 0
91         if len(list_of_args) % 2 == 1:
92             remaining_length = (len(list_of_args) - 1) // 2
93         if len(list_of_args) % 2 == 0:
94             remaining_length = len(list_of_args) // 2
95         if remaining_length % 2 != 0:
96             first_position = remaining_length // 2
97             second_position = (len(list_of_args) - 1) - first_position
98             q1 = list_of_args[first_position]
99             q3 = list_of_args[second_position]
100             result = q3 - q1
101             return result
102         if remaining_length % 2 == 0:
103             second_position = remaining_length // 2
104             first_position = second_position - 1
105             q1 = (list_of_args[first_position] + list_of_args[second_position]) / 2
106             third_position = len(list_of_args) - second_position
107             forth_position = third_position - 1
108             q3 = (list_of_args[forth_position] + list_of_args[third_position]) / 2
109             result = q3 - q1
110             return result
111     else:
112         return "Такой операции не существует"
113     result = round(result, precision)
114     return result

```

5. Пример результата работы программы:

```

Введите первое число:
1
Введите второе число:
2
Введите тип операции:
medium
Введите точность вычисления (по умолчанию 1e-6)

Введите дополнительные числа через запятую(при надобности)
1,2,1,2
Ответ: 1.5

```

Задание 2.

1. Формулировка:

Реализуйте программно классическую простую игру "угадай число" (guess number) с помощью алгоритма медленного перебора (инкремента) по одному числу, либо с помощью алгоритма бинарного поиска. Алгоритм принимает на вход само число, которое он должен угадать, интервал значений в котором оно загадано и в цикле делает угадывания тем или иным выбранным вами способом. После угадывания из функции алгоритма возвращается угаданное число и число угадываний/сравнений, которые пришлось проделать. Обязательно напишите хорошую документацию к своему коду.

2. Математическая модель.

3. Список идентификаторов.

Имя	Тип	Смысл
x	int	Входная переменная
y	int	Входная переменная
z	int	Входная переменная
low	int	Параметр функции guess_number
up	int	Параметр функции guess_number
number	int	Параметр функции guess_number
count	int	Результат функции guess_number
i	int	Результат функции guess_number

4. Код программы.

```
1 usage
2 def guess_number(number, low, up):
3     """
4     Arguments:
5         number -- number that the program should guess
6         low -- lower bound of an interval
7         up -- upper bound of an interval
8
9     Return the guessed number and a number of guessing attempts
10    """
11    count = 0
12    for i in range(low, up, 1):
13        count += 1
14        if i == number:
15            print(f"Ваше число: {x}! Количество попыток угадывания - {count}")
16            break
17
18    print("Введите число")
19    x = int(input())
20    print("Введите нижнюю границу интервала, в котором загадано число")
21    y = int(input())
22    print("Введите верхнюю границу интервала, в котором загадано число")
23    z = int(input())
24
25    guess_number(x, y, z)
26
```


5. Результат работы программы.

Введите число

15

Введите нижнюю границу интервала, в котором загадано число
-100

Введите верхнюю границу интервала, в котором загадано число
16

Ваше число: 15! Количество попыток угадывания - 116